Assignment 2: Bomb Lab

System Programming

CSE306

Instructor:

Prof. Yongwoo Lee (yongwoo@dgist.ac.kr)

Assignment 2 – Bomb Lab is available!

- **Due : Fri 3rd Apr, 11:00 PM**

- Covered topics in Bomb Lab
  - x86_64 assembly
  - + GDB

- PLEASE READ bomblab.pdf CAREFULLY (especially HINT section!)
- Start your assignment early!
- If you have some problem, please ask to TA.

Check your bomb

- ***Dr.Evil*** has already installed the bomb into your server account

- When unzipping bomb.tar, you get:
  - bomb : bomb executable
  - bomb.c : source code for the main bomb routine
  - README : file contain bomb number and student ID

- Every bomb has owner
- **check the README file whether described student ID is yours!**

About Bomb

- 6 phases
  - Give right answers to each phase.
  - 1-4: 10 points, 5-6: 15 points
  - Autograding!
    - Scoreboard: http://london.coruslab.or.kr:8081/scoreboard
- You cannot run the bomb outside the Class server; otherwise, the bomb will be exploded.
- Each bomb is different! You cannot use your friend's solution.

Detonating your bomb

- Blowing up your bomb automatically notifies auto grading server
  - The server takes 0.5 of your points each time.
  - However, It's very easy to prevent explosions using break points in GDB

- Inputting the correct string moves you to the next phase.
- Solving the phases in order they are given.
- Finishing a phase also notifies auto grading server.

Bomb Hints

- ***Dr. Evil*** may be evil, but he isn't cruel. You may assume the function's role from their name.
  - Ex1) phase_1() : most likely the first phase
  - Ex2) explode_bomb() : most likely called for exploding

- Use the man pages for library functions.
  - You can examine the assembly like snprintf()
  - But man snprintf() is much easier

Bomb Hints

- **`You need to using Debugger!`**
  - Help to examine while stepping through your program:
    - registers
    - the stack
    - Contents of program memory
    - instruction stream

GDB

- **G**NU **D**e**b**ugger(=gdb) is one of the most common used debugger.
  - Let you inspect your program as it's executing.

- You can open gdb by typing $gdb into shell:
  - $gdb bomb

basic GDB command

b function	Set a breakpoint at the beginning of function
b line_number	Set a breakpoint at line number of the current file
info b	List all breakpoints
delete n	Delete breakpoint number n
r args	Start the program being debugged, possibly with command line arguments args.
s count	Single step the next count statments (default is 1). Step into functions.
n count	Single step the next count statments (default is 1). Step over functions.
finish	Execute the rest of the current function. Step out of the current function.
c	Continue execution up to the next breakpoint or until termination if no breakpoints are encountered.
p expression	print the value of expression
set listsize n	Set the number of lines listed by the list command to n
l optional_line	List next listsize lines. If optional_line is given, list the lines centered around optional_line.
backtrace(=where)	Display the current line and function and the stack of calls that got you there.
disassemble	Disassemble a specified section of memory
h optional_topic	help or help on optional_topic
q	quit gdb

Helpful GDB command (1/3)

- `run(=r) <args>`
  - Run program with args `<args>`
  - Convenient for using answer text file.

- `break(=b) <location>`
  - Stop execution of program when it reaches certain point
  - `<location>`
    - Function name or function name + 4.
    - Actual address with `*0xXXX…`
    - `Line number`

- Navigate through assembly:
  - `step(=s) <count> / next(=n) <count>`
    - `execute as many <count> lines of code.`
    - Steps into function / step over function
  - continue(=c)
    - Execute until next breakpoint is hit

Helpful GDB command (2/3)

- `info`
  - Info Register : Print hex values in every register
  - info stack : print backtrace of stack
  - Info breakpoint : print status of breakpoint

```
(gdb) info register
rax            0x0      0
rbx            0x0      0
rcx            0x5555555546d0   93824992233168
rdx            0x7fffffffe3f8   140737488348152
rsi            0x7fffffffe3e8   140737488348136
rdi            0x555555554754   93824992233300
rbp            0x7fffffffe300   0x7fffffffe300
rsp            0x7fffffffe2e8   0x7fffffffe2e8
r8             0x7ffff7dd0d80   140737351847296
r9             0x7ffff7dd0d80   140737351847296
r10            0x2      2
r11            0x7      7
r12            0x555555554540   93824992232768
r13            0x7fffffffe3e0   140737488348128
r14            0x0      0
r15            0x0      0
rip            0x7ffff7a48e80   0x7ffff7a48e80 <__printf>
eflags         0x206    [ PF IF ]
cs             0x33     51
ss             0x2b     43
ds             0x0      0
es             0x0      0
fs             0x0      0
gs             0x0      0
```

```
(gdb) info stack
#0  __printf (format=0x555555554754 "hello world") at printf.c:28
#1  0x000055555555469f in main () at a.c:10
```

Helpful GDB command (3/3)

- `Print(=p) (/x or /d) register or address`
  - Print hex or decimal contents of register or address

- `x $register or 0xaddress`
  - Prints what is in the register or at the given address

```
(gdb) p $rsp
$3 = (void *) 0x7fffffffe2c0
(gdb) x $rsp
0x7fffffffe2c0: 0x00000001
```

More hint!

- **`objdump -t bomb`** examines the symbol table.
- **`objdump -d bomb`** disassembles all bomb code.
- **`strings bomb`** prints all printable strings.

- Save the `stdout` to a file: `>`
  - Ex) `objdump -d bomb > bomb.s`

Before starting

- ***PLEASE READ bomblab.pdf CAREFULLY***
- GDB: http://heather.cs.ucdavis.edu/~matloff/UnixAndC/CLanguage/Debug.html
- `man gdb, man snscanf, man objdump`
- Feel free to ask to TA!
  - The TA won't dissolve your bomb,
    but you can get some infomation how to use GDB.